

Model Checking

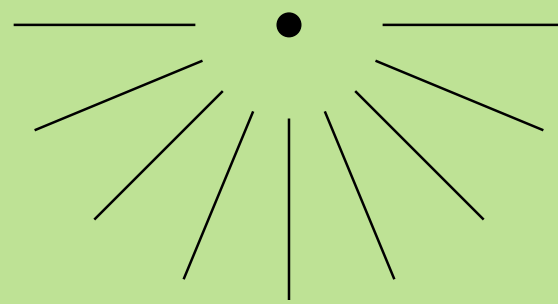
Alunos: Guilherme Piacentini da Silva e Matheus Lopes de Freitas

Disciplina: Métodos Formais

Data: 18/09/2025

BCC-UDESC





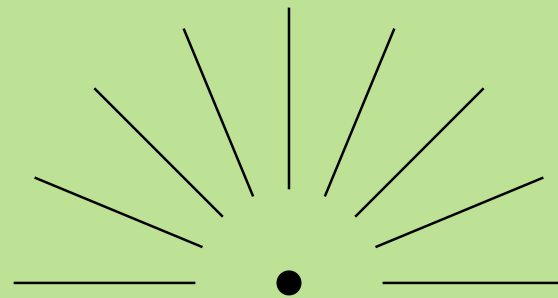
Introdução ao Model Checking

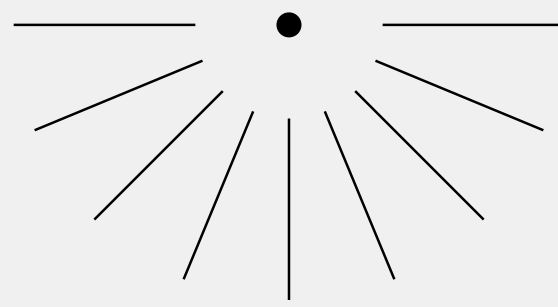
Definição: Verificação automática de que um modelo formal de um sistema satisfaz uma propriedade formal especificada.

- **Entrada:**

- 1) Um modelo do sistema (ex: autômato, Kripke structure).
- 2) Uma fórmula lógica descrevendo a propriedade desejada (ex: "nunca deadlock").

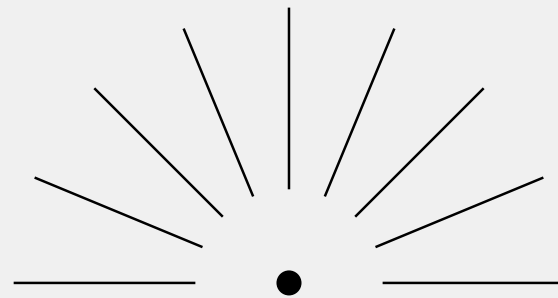
- **Saída:** True (se a propriedade vale) ou False + contra-exemplo (se não vale).

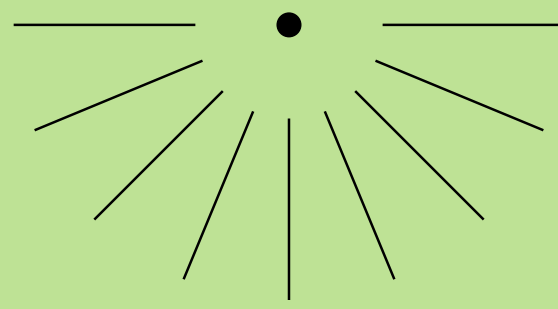




Por que usar Model Checking?

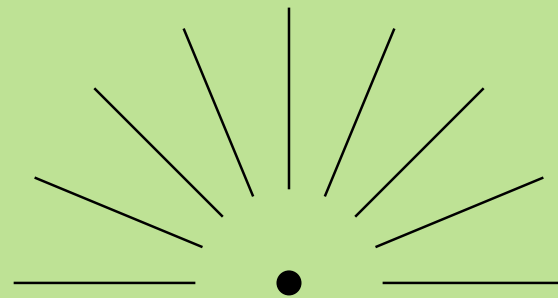
- **Verificação Exaustiva:** Verifica todas as possíveis execuções do sistema.
- **Garantia:** Se bem implementados é mais completo que testar manualmente.
- **Contra-Exemplos:** Quando falha, fornece um caminho de execução que viola a propriedade, auxiliando no debug.

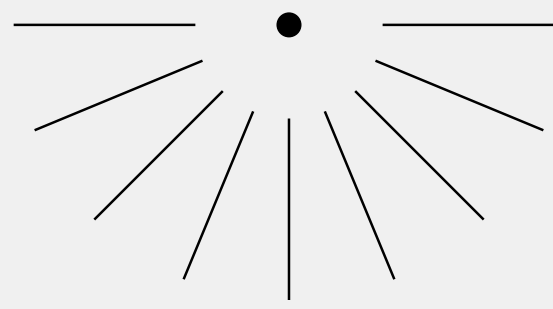




Conceitos Fundamentais

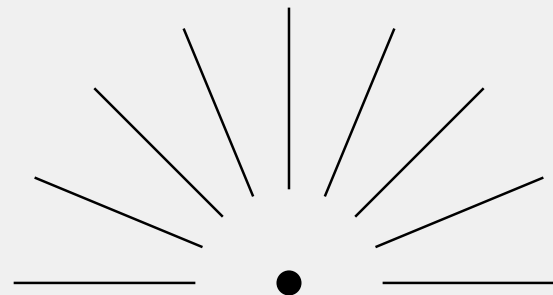
- **Sistema de Transição de Estados (Kripke Structure):**
 - 1) Estados: Configurações possíveis do sistema.
 - 2) Transições: Mudanças de um estado para outro.
 - 3) Rótulos (Labels): Propriedades atômicas verdadeiras em cada estado.
- **Propriedades:** Expressas em Lógica Temporal.

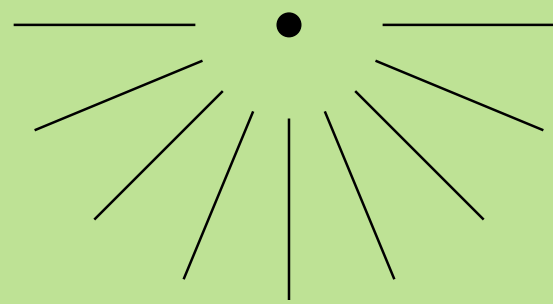




Lógica Temporal Computacional (CTL)

- **Foca em Árvores de Computação:** Propriedades de ramificação do tempo.
- **Quantificadores de Caminho:**
 - 1) A (For All paths): Para todo caminho...
 - 2) E (There Exists a path): Existe um caminho...





Lógica Temporal Computacional (CTL) [2]

- **Operadores Temporais:**

1) F (Finally): Eventualmente.

2) G (Globally): Sempre.

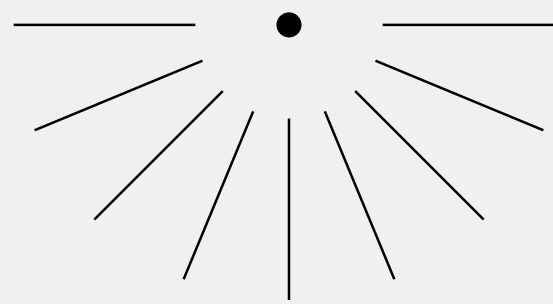
3) X (Next): No próximo estado.

4) U (Until): Até que.

- **Exemplo (CTL):** $AG (request \rightarrow AF response)$

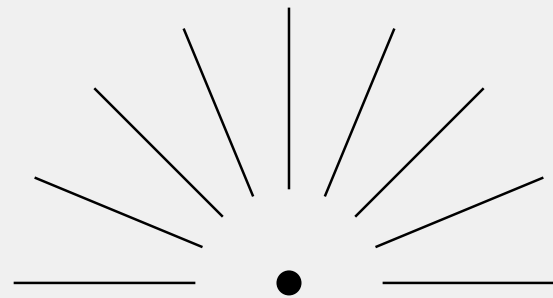
"Sempre, se houver uma requisição, eventualmente haverá uma resposta."

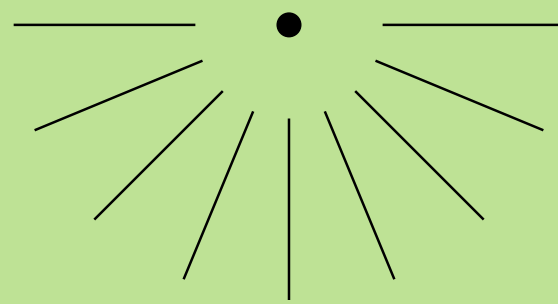




Lógica Temporal Linear (LTL)

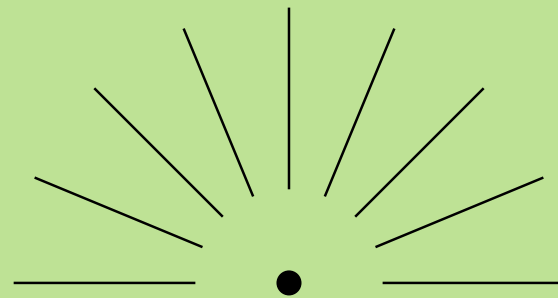
- **Foca em Caminhos Individuais:** Propriedades lineares do tempo.
- **Operadores Temporais:** Usa os mesmos operadores (F, G, X, U).
- Não possui quantificadores A e E. As fórmulas são implicitamente sobre todos os caminhos.
- **Exemplo (LTL):** $G(\text{request} \rightarrow F \text{ response})$
"Sempre, se houver uma requisição, eventualmente haverá uma resposta."

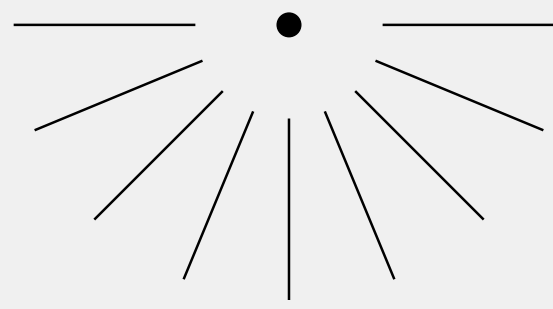




O Algoritmo de Verificação

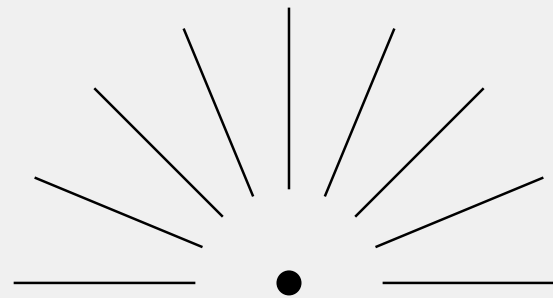
- **Modelagem:** O sistema é modelado como uma estrutura de Kripke (M).
- **Especificação:** A propriedade é expressa como uma fórmula lógica temporal (φ).
- **Verificação:** O model checker verifica se $M \models \varphi$ (se o modelo M satisfaz φ).
- **Resultado:**
 - se SIM, o sistema está correto para aquela propriedade.
 - se NÃO, o model checker produz um contra-exemplo (traço de execução que leva à violação).

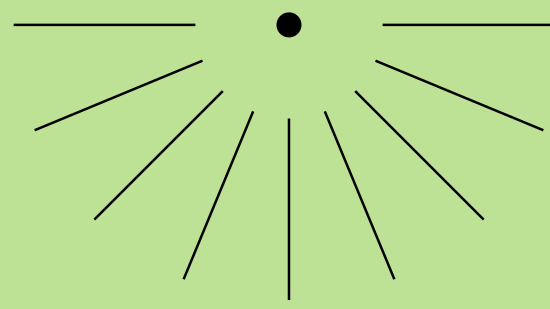




Model Checkers Populares

- **SPIN:** Verificador para sistemas concorrentes (usando LTL).
Linguagem de modelagem: PROMELA.
- **NuSMV:** Verificador para sistemas síncronos e assíncronos (suporta CTL e LTL).
- **UPPAAL:** Especializado em sistemas de tempo real.
- **TLA+:** Linguagem de especificação e model checker desenvolvida por Leslie Lamport.



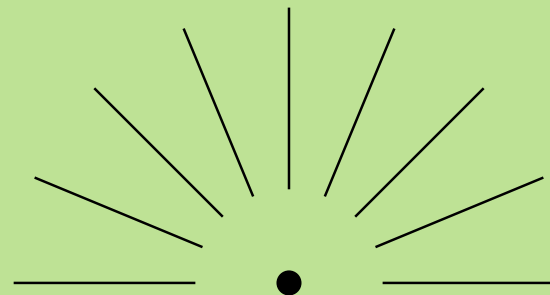


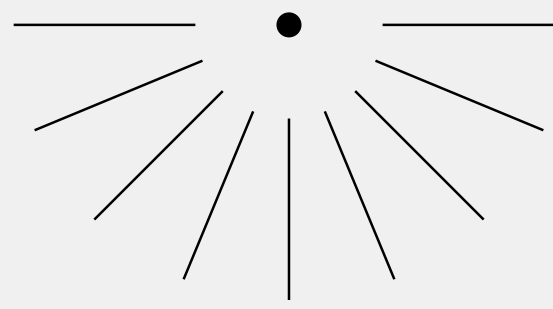
Exemplo de uso: Semáforo

- **Sistema:** Controle de um cruzamento simples.
- **Propriedade de Segurança (CTL):** $AG \neg(\text{verde_norte} \wedge \text{verde_leste})$

"É sempre verdade que os semáforos norte e leste **NUNCA** estarão verdes ao mesmo tempo."

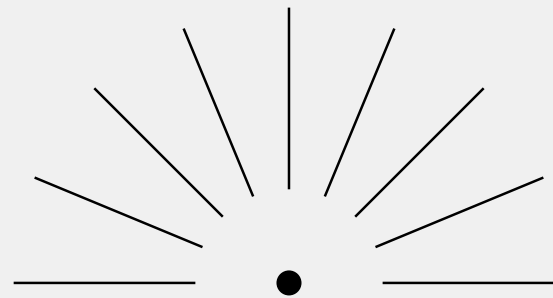
- **Para verificar:** O model checker explora todos os estados possíveis do semáforo e checa se em nenhum deles essa condição $(\text{verde_norte} \wedge \text{verde_leste})$ é verdadeira.

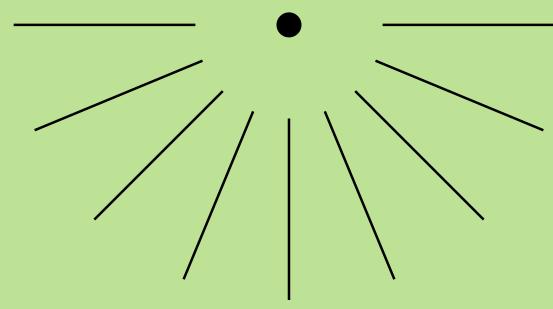




Desafios e Limitações

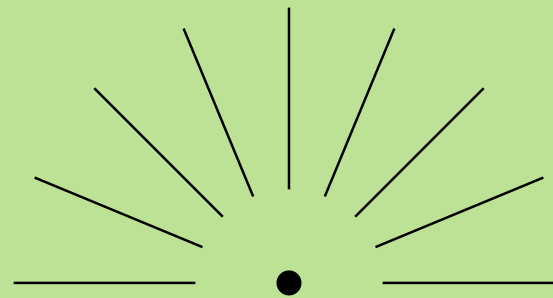
- **Complexidade de Modelagem:** Criar um modelo preciso pode ser difícil e demorado.
- **Abstração:** Muitas vezes é necessário abstrair detalhes do sistema real para implementar.
- **O Grande Desafio:** State Space Explosion (Explosão Combinatória do Espaço de Estados).

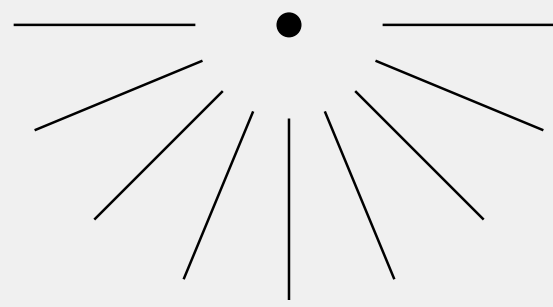




State Space Explosion

- **O Problema:** O número de estados de um sistema cresce exponencialmente com o número de variáveis/components.
- **Exemplo:** Um sistema com 10 variáveis booleanas tem $2^{10} = 1024$ estados. Com 20 variáveis, $2^{20} = 1.048.576$ estados.
- **Consequência:** Pode esgotar a memória disponível, tornando a verificação impraticável.





Técnicas de Mitigação

- Model Checking Simbólico: Usa BDDs (Binary Decision Diagrams) para representar conjuntos de estados de forma compacta.
- Model Checking por Abstraction: Cria um modelo simplificado (abstract) do sistema que preserva as propriedades a serem verificadas.
- Partial Order Reduction: Explora a independência entre eventos concorrentes para evitar verificar caminhos redundantes.
- **Bounded Model Checking:** Limita a verificação a um número finito de passos (útil para encontrar bugs).



Model Checking vs. Teste

Característica	Teste	Model Cheching
Cobertura	Amostrai (parcial)	Exaustiva (completa)
Automação	Parcial (scripts)	Alta
Contra-Exemplo	Só mostra a falha	Mostra caminho até a falha
Escalabilidade	Mais simples de escalar	State Space Explosion
Fase de Uso	Pós-implementação	Pré-implementação

Fim!